EJERCICIO 14

Brote de gripe en una residencia

MODELADO

Qué representa cada cosa:
- Vértices: los residentes.
- Aristas: un contacto estrecho entre dos residentes.
- Tipo de grafo: NO DIRIGIDO. Si Ana tuvo contacto estrecho con Beto, Beto lo tuvo con Ana, y el contagio va en los dos sentidos. No hace falta ponderarlo: todos los contagios tardan un día. No es conexo: Lu, Mati y Nico forman un grupo aparte.

Estructuras elegidas y por qué:
- contactos: grafo no dirigido con lista de adyacencias. La red tiene pocos contactos (14 de 78 posibles) y solo hace falta recorrer los contactos de cada residente.
- Día de contagio de cada residente: un diccionario residente, día. Guarda el resultado y sirve para saber si un residente ya se contagió (si no tiene entrada, todavía no). Conserva el orden en que se contagian.
- Cola: guarda a los contagiados que todavía no contagiaron a sus contactos. Procesa primero a los de menor día, así cada residente recibe el día más temprano.
- residentes: lista en orden de registro (para recorrer todos los residentes).
- Para los puentes: dos diccionarios (bp y bajo) y un conjunto con el resultado, sin repetidos.

simularBrote(pacientesCero, dias)

Lenguaje natural:
Es una búsqueda en amplitud con varios orígenes. Los pacientes cero que están en la red se contagian el día 0, se anotan en el diccionario y entran a la cola. Se saca un residente de la cola: si su día es menor que "dias", a cada contacto que todavía no tiene día se le anota el día siguiente y entra a la cola. Si su día es igual a "dias" no contagia a nadie, porque sus contactos se contagiarían después del límite. Como la cola atiende los días en orden, cada residente recibe el día más temprano y entra una sola vez.

Precondiciones: No hay.

Postcondiciones:
- El resultado tiene solo residentes contagiados hasta el día "dias" inclusive, cada uno con su día de contagio (los pacientes cero con día 0).
- Los pacientes cero que no están en la red se ignoran.
- Si dias < 0 o no hay pacientes cero válidos, el resultado está vacío.
- La red no cambia.

Pseudocódigo:

```
simularBrote(pacientesCero, dias) devuelve Diccionario de residente -> día
Comienzo
    dia <- diccionario vacío que conserva el orden de ingreso
    SI pacientesCero ES NULO OR dias < 0 ENTONCES
        devolver dia
    FIN SI

    cola <- cola vacía
    PARA CADA p EN pacientesCero HACER
        SI existeVertice(contactos, p) AND p NO ESTA EN dia ENTONCES
            dia[p] <- 0
            cola.encolar(p)
        FIN SI
    FIN PARA

    MIENTRAS cola NO ESTA VACIA HACER
        u <- cola.desencolar()
        d <- dia[u]
        SI d < dias ENTONCES
            PARA CADA arista EN adyacencias(contactos, u) HACER
                v <- arista.destino
                SI v NO ESTA EN dia ENTONCES
                    dia[v] <- d + 1
                    cola.encolar(v)
                FIN SI
            FIN PARA
```

```
        FIN SI
    FIN MIENTRAS
    devolver dia
Fin
```

Orden del tiempo de ejecución:
Cada residente entra a la cola una vez y cada contacto se mira dos veces: O(V + E).

residentesPuente()

Lenguaje natural:
Un residente "separa a otros" si al aislarlo hay residentes que dejan de estar conectados entre sí: es un PUNTO DE ARTICULACIÓN del grafo de contactos. Se usa el mismo algoritmo del ejercicio 11: búsqueda en profundidad numerando a los residentes (bp) y calculando su "bajo". Un residente que no es la raíz es puente si tiene un hijo con bajo >= bp de él. La raíz es puente solo si tiene 2 o más hijos. Si hay varios grupos desconectados, la búsqueda se repite desde cada residente que todavía no se visitó.

Precondiciones: No hay.

Postcondiciones:
- Devuelve una lista sin repetidos con los residentes que, al aislarlos (quitarlos con sus contactos), aumentan la cantidad de grupos de residentes conectados entre sí.
- La red no cambia.

Pseudocódigo: es el del ejercicio 11 (ver Ejercicio11.txt):

```
residentesPuente() devuelve Lista
Comienzo
    bp <- diccionario vacío ; bajo <- diccionario vacío ; puentes <- lista vacía sin repetidos
    PARA CADA r EN residentes HACER
        SI r NO ESTA EN bp ENTONCES
            bp[r] <- cantidad de claves de bp + 1
            bajo[r] <- bp[r]
            hijos <- 0
            PARA CADA arista EN adyacencias(contactos, r) HACER
                SI arista.destino NO ESTA EN bp ENTONCES
                    hijos <- hijos + 1
                    bpf(arista.destino, r)
                FIN SI
            FIN PARA
            SI hijos >= 2 ENTONCES
                puentes.agregar(r)              // r es la raíz de este grupo
            FIN SI
        FIN SI
    FIN PARA
    devolver puentes
Fin

bpf(u, padre)
Comienzo
    bp[u] <- cantidad de claves de bp + 1
    bajo[u] <- bp[u]
    PARA CADA arista EN adyacencias(contactos, u) HACER
        v <- arista.destino
        SI v = padre ENTONCES continuar         // la arista por la que se llegó
        SI v ESTA EN bp ENTONCES
            bajo[u] <- minimo(bajo[u], bp[v])   // contacto que vuelve atrás
        SINO
            bpf(v, u)
            bajo[u] <- minimo(bajo[u], bajo[v])
            SI bajo[v] >= bp[u] ENTONCES
                puentes.agregar(u)              // u no es la raíz: bpf nunca se llama con la raíz
            FIN SI
        FIN SI
    FIN PARA
Fin
```

Orden del tiempo de ejecución: O(V + E).

3 RESPUESTAS A LAS PREGUNTAS DEL ENUNCIADO

PREGUNTA 1: ¿Qué día se contagia cada residente si Ana y Juan son los pacientes cero y se simulan 10 días?

Cola: Ana, Juan (día 0).
Día 0: Ana y Juan.
Día 1: contactos de Ana: Beto, Caro. Contactos de Juan: Hugo.
Día 2: contactos de Beto: ninguno nuevo. De Caro: Dani. De Hugo: Gabi e Ines.
Día 3: contactos de Dani: Eli y Fede. De Gabi: Fede, que ya estaba anotado. De Ines: ninguno nuevo.
Día 4 en adelante: nadie nuevo. Lu, Mati y Nico no tienen contacto con los demás.

Residente	Día de contagio
Ana	0
Juan	0
Beto	1
Caro	1
Hugo	1
Dani	2
Gabi	2
Ines	2
Eli	3
Fede	3
Lu	no se contagia
Mati	no se contagia
Nico	no se contagia

PREGUNTA 2: ¿Qué residentes separan a otros si se los aísla?
Caro, Dani, Fede, Gabi, Hugo y Mati.
- Caro: sin Caro, Ana y Beto quedan separados del resto.
- Dani: sin Dani, Ana, Beto y Caro quedan separados de Eli, Fede, Gabi, Hugo, Ines y Juan.
- Fede: sin Fede, Gabi, Hugo, Ines y Juan quedan separados de Ana, Beto, Caro, Dani y Eli.
- Gabi: sin Gabi, Hugo, Ines y Juan quedan separados del resto.
- Hugo: sin Hugo, Juan queda aislado.
- Mati: sin Mati, Lu y Nico quedan separados entre sí.

No separan a nadie: Ana y Beto (están con Caro en un triángulo), Eli (triángulo con Dani y Fede), Ines (triángulo con Gabi y Hugo), y Juan, Lu y Nico (cada uno tiene un solo contacto).

PARTE 2 - CORRECCIÓN DE CÓDIGO

simularBrote

Línea 16: String v = e.source();
Qué está mal: source() es el mismo residente u, no su contacto. Entonces v siempre es u, que ya está en visitados, y nadie se contagia. Con Ana y Juan como pacientes cero el resultado tiene solo a Ana y a Juan con día 0.
Corrección: String v = e.target();

Líneas 3 y 12: se usa una pila (TDAPila) en lugar de una cola.
Qué está mal: el contagio avanza un día por vez, es una búsqueda en amplitud y necesita una cola. Con la pila, un residente recibe el día de la primera vez que se lo alcanza, que puede ser por un camino más largo. Con Ana y Juan como pacientes cero (y la línea 16 corregida), Caro queda en el día 5 y Beto en el 6, cuando lo correcto es el día 1 para los dos.
Corrección: usar una cola: encolar y desencolar en lugar de mete y saca.

Línea 14: if (d <= dias).
Qué está mal: un residente con día igual a dias contagiaría a sus contactos en el día dias + 1, que está fuera del límite. Con dias = 1, Dani, Gabi e Ines quedarían contagiados el día 2.
Corrección: if (d < dias).

Líneas 4, 9, 17 y 20: visitados es una lista y contiene() la recorre.
Qué está mal: preguntar si un residente ya se contagió recorre toda la lista, y el enunciado pide no hacerlo.
Corrección: eliminar visitados y preguntar con dia.buscar(v) == null. Se quitan las líneas 4, 9 y 20, y la línea 17 pasa a ser if (dia.buscar(v) == null).

Líneas 5 a 10: no se controla el paciente cero.
Qué está mal: los pacientes cero que no están en la red se tienen que ignorar y acá se insertan igual (con "Zoe", el resultado incluiría a Zoe con día 0). Si un paciente cero está repetido, entra dos veces a la pila.
Corrección: dentro del for, agregar la condición
```
if (contactos.existeVertice(p) && dia.buscar(p) == null) { dia.insertar(p, 0); pendientes.mete(p); }
```
y quitar visitados.agregar(p).

residentesPuente

Línea 33: bpf(residentes.obtener(0), null);
Qué está mal: la búsqueda se hace solo desde el primer residente, así que solo se recorre su grupo. Lu, Mati y Nico quedan sin visitar y Mati, que separa a Lu de Nico, no aparece en el resultado. Si no hay residentes, obtener(0) falla.
Corrección: recorrer todos los residentes y buscar desde cada uno que todavía no se visitó:
```
for (int i = 0; i < residentes.tamaño(); i++) {
    String r = residentes.obtener(i);
    if (bp.buscar(r) == null) bpf(r, null);
}
```

Líneas 46 a 48: la raíz siempre se agrega como puente.
Qué está mal: bajo[v] >= bp[u] siempre se cumple para la raíz, así que se agrega aunque no separe a nadie. La raíz es puente solo si tiene 2 o más hijos. Ana es la primera residente, o sea la raíz, y tiene un solo hijo (Beto): el código la informa como puente y aislarla no separa a nadie.
Corrección: para la raíz (padre == null) contar sus hijos y agregarla solo si son 2 o más. Para el resto se mantiene la condición:
```
if (padre != null && bajo.buscar(v) >= bp.buscar(u)) puentes.agregar(u);
```
(para padre == null: hijos++, y al final if (hijos >= 2) puentes.agregar(u))